JAVA INTERVIEW PREPARATION

CHAPTER 6

IMMUTABILITY AND
DEFENSIVE COPYING

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Immutability and Defensive Copying in Java

Senior / Lead Java Developer Reading Chapter

An immutable object does not change its observable state after construction. That simple definition has far-reaching consequences: immutable values are easier to reason about, safer to share between threads, stable as map keys, simpler to cache, and less vulnerable to accidental modification through distant references.

Immutability is not achieved merely by removing setters or adding `final` to fields. A class can contain final references to mutable collections, arrays, dates, or domain objects. If callers can change those referenced objects, the containing object is only superficially immutable.

Senior-level design requires reasoning about the complete reachable object graph, construction and publication, ownership, serialization boundaries, and whether a snapshot or a live view is intended.

Object Immutability Versus Reference Reassignment

An immutable object cannot change, but a variable can be reassigned to another immutable value.

```
Money total = Money.of("10.00", "USD");
total = total.add(Money.of("5.00", "USD"));
```

The original Money object remains unchanged. `add()` returns another value, and the variable is updated to reference it.

```
Before add

total -------> Money(10.00 USD)

After assignment

total -------> Money(15.00 USD)

               Money(10.00 USD) remains unchanged
```

This functional update style makes state transitions explicit. A caller holding the original reference continues to observe the original value.

A Basic Immutable Value Object

An immutable value should establish its complete valid state in one construction operation. Callers should not need to create an incomplete shell and then remember a sequence of setters. Operations on the value should either return derived information or create another valid value.

Consider a money value:

```
public final class Money {

    private final BigDecimal amount;
    private final Currency currency;

    public Money(BigDecimal amount, Currency currency) {
        Objects.requireNonNull(amount, "amount");
        Objects.requireNonNull(currency, "currency");

        this.amount = amount.stripTrailingZeros();
        this.currency = currency;
    }

    public BigDecimal amount() {
        return amount;
    }

    public Currency currency() {
        return currency;
    }

    public Money add(Money other) {
        requireSameCurrency(other);
        return new Money(amount.add(other.amount), currency);
    }
}
```

The class is final, its fields are private and final, construction validates all required state, and operations return new values instead of changing fields.

`BigDecimal` is immutable and `Currency` behaves as an immutable value, so returning their references does not expose a mutation path. If a field referred to a mutable type, accessors would need a different strategy.

The class should also implement value equality and hashing from its stable state. Immutability makes the hash code stable for the object's lifetime.

Why `final` Fields Are Not Enough

A final field prevents the reference from being reassigned. It does not freeze the object being referenced.

```
public final class Schedule {

    private final List<Instant> times;

    public Schedule(List<Instant> times) {
        this.times = times;
    }

    public List<Instant> times() {
        return times;
    }
}
```

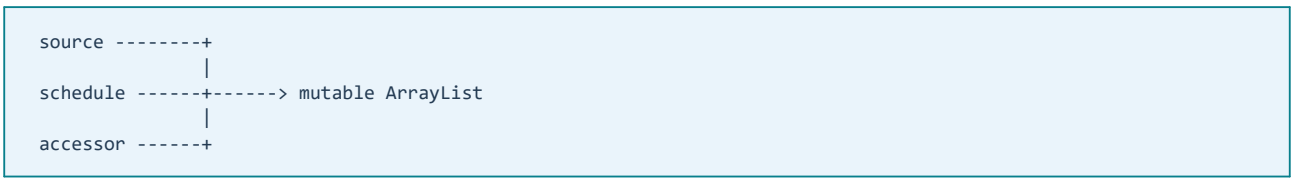
The reference stored in `times` is final, but both the constructor caller and accessor caller can modify the same List.

```
List<Instant> source = new ArrayList<>();
source.add(first);

Schedule schedule = new Schedule(source);

source.add(second);          // schedule changes from the outside
schedule.times().clear();    // accessor exposes internal state
```

The object graph looks like:



The problem is aliasing: several references share ownership of one mutable object.

Defensive Copying on Input and Output

A defensive copy gives the immutable object its own protected representation.

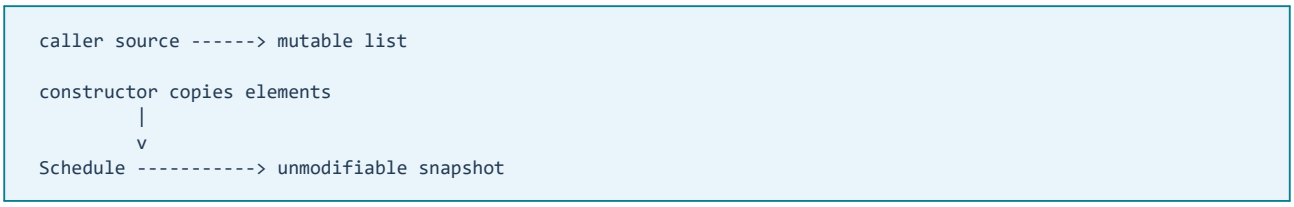
```
public final class Schedule {

    private final List<Instant> times;

    public Schedule(List<Instant> times) {
        this.times = List.copyOf(times);
    }

    public List<Instant> times() {
        return times;
    }
}
```

`List.copyOf()` creates an unmodifiable snapshot when necessary and rejects null elements. Later changes to the source list do not affect the Schedule. Because the stored list is unmodifiable, returning it is safe as long as its elements are also immutable.



An unmodifiable view is different from a copy:

```
this.times = Collections.unmodifiableList(times);
```

The wrapper prevents mutation through that reference, but changes made through the original `times` reference remain visible. It is a read-only view of live mutable state, not an independent snapshot.

Use a copy when the object owns a snapshot. Use an unmodifiable view only when live visibility is intentional and the underlying mutation is controlled elsewhere.

Shallow and Deep Immutability

Collection copying is usually shallow. The collection structure is protected, but the elements are the same references.

```
public final class Team {

    private final List<Member> members;

    public Team(List<Member> members) {
        this.members = List.copyOf(members);
    }
}
```

If `Member` is mutable, a caller can still alter the Team's observable state:

```
Member member = new Member("Alice");
Team team = new Team(List.of(member));

member.rename("Bob"); // team now appears to contain Bob
```

Deep immutability requires every reachable object affecting observable state to be immutable or privately copied.

```
public Team(List<Member> members) {
    this.members = members.stream()
        .map(Member::immutableCopy)
        .toList();
}
```

Deep copying can be expensive and may be impossible when elements represent resources or shared entities. The better design may be to store immutable `MemberSnapshot` values rather than mutable domain entities.

Immutability should follow ownership and domain meaning, not a mechanical demand to recursively clone every object.

Arrays Require Special Care

Arrays are mutable and cannot be made unmodifiable with a wrapper.

```
public final class Digest {

    private final byte[] bytes;

    public Digest(byte[] bytes) {
        this.bytes = bytes;
    }

    public byte[] bytes() {
        return bytes;
    }
}
```

Both boundaries leak mutation. A correct implementation copies on input and output:

```
public final class Digest {

    private final byte[] bytes;

    public Digest(byte[] bytes) {
        this.bytes = bytes.clone();
    }

    public byte[] bytes() {
        return bytes.clone();
    }
}
```

Returning a copy allocates on every call. If callers only need comparison, encoding, or streaming, expose those operations instead of the raw representation.

```
public String hexadecimal() {
    return HexFormat.of().formatHex(bytes);
}
```

This keeps ownership inside the class and can provide a cached derived value if profiling justifies it.

Legacy Mutable Date Types

`java.util.Date` is mutable despite commonly representing a value.

```
public final class Contract {

    private final Date effectiveAt;

    public Contract(Date effectiveAt) {
        this.effectiveAt = effectiveAt;
    }

    public Date effectiveAt() {
        return effectiveAt;
    }
}
```

A caller can change the stored Date through either reference. Defensive copies are required if the legacy type must be used:

```
public Contract(Date effectiveAt) {
    this.effectiveAt = new Date(effectiveAt.getTime());
}

public Date effectiveAt() {
    return new Date(effectiveAt.getTime());
}
```

Prefer immutable `java.time` types such as `Instant`, `LocalDate`, `OffsetDateTime`, or `ZonedDateTime`, selected according to the domain semantics. Choosing the right time type is as important as immutability: a birthday is normally a `LocalDate`, while an audit timestamp is normally an `Instant`.

Records Are Only Shallowly Immutable

A record makes its component fields private and final and generates value methods, but it does not defensively copy mutable components.

```
public record OrderDraft(List<LineItem> items) {
}
```

The caller's list remains aliased. A compact constructor can protect it:

```
public record OrderDraft(List<LineItem> items) {

    public OrderDraft {
        items = List.copyOf(items);
    }
}
```

This is deeply immutable only when `LineItem` is immutable. Records are excellent for transparent value aggregates, but the record keyword is not a substitute for ownership analysis.

Arrays are especially surprising record components because generated equality uses the array's identity-based `equals()`. A dedicated immutable wrapper may provide better copying and content equality.

Builders and Safe Construction

An immutable object may require many optional inputs or multi-step validation. A mutable builder can collect construction data and produce one immutable snapshot.

```
public final class ReportRequest {

    private final Instant from;
    private final Instant to;
    private final Set<String> regions;

    private ReportRequest(Builder builder) {
        if (builder.from == null || builder.to == null) {
            throw new IllegalStateException("Time range is required");
        }

        if (builder.from.isAfter(builder.to)) {
            throw new IllegalArgumentException("Invalid time range");
        }

        this.from = builder.from;
        this.to = builder.to;
        this.regions = Set.copyOf(builder.regions);
    }

    public static final class Builder {
        private Instant from;
        private Instant to;
        private final Set<String> regions = new HashSet<>();

        public Builder from(Instant value) {
            this.from = value;
            return this;
        }

        public Builder to(Instant value) {
            this.to = value;
            return this;
        }

        public Builder addRegion(String value) {
            regions.add(value);
            return this;
        }

        public ReportRequest build() {
            return new ReportRequest(this);
        }
    }
}
```

The builder is mutable and should generally be thread-confined. The constructed object copies mutable builder state, so later builder reuse does not modify previous results.

Avoid allowing `this` to escape from the immutable object's constructor by registering callbacks, publishing to a static collection, or starting a thread. Other code could observe partially initialized state.

Final Fields and Safe Publication

The Java Memory Model gives final fields special initialization guarantees. When an object is correctly constructed and does not escape during construction, threads obtaining the reference through ordinary publication mechanisms receive strong guarantees for the initialized final fields and objects reachable through them as established during construction.

Final fields do not make mutable reachable objects thread-safe. If a final field references an ArrayList that is changed after construction, concurrent access still requires synchronization or confinement.

```
constructor
   |
   +-- validate all inputs
   +-- copy mutable state
   +-- assign final fields
   +-- do not publish this
   |
construction completes
   |
   v
safely published immutable value
```

Immutability reduces synchronization because readers do not coordinate state changes. Publication of the reference still matters. Common safe publication mechanisms include final fields of already safely published objects, static initialization, volatile references, locks, concurrent collections, and executor handoff.

Functional Updates and Persistent Style

Immutable objects represent updates by returning another object.

```
public record CustomerProfile(
        CustomerId id,
        String displayName,
        Set<Role> roles) {

    public CustomerProfile {
        roles = Set.copyOf(roles);
    }

    public CustomerProfile withDisplayName(String newName) {
        return new CustomerProfile(id, newName, roles);
    }

    public CustomerProfile addRole(Role role) {
        Set<Role> updated = new HashSet<>(roles);
        updated.add(role);
        return new CustomerProfile(id, displayName, updated);
    }
}
```

This approach is simple and reliable for small values. Copying a large collection for every update may be expensive. Persistent collection libraries use structural sharing so versions reuse unchanged internal nodes.

The decision should consider update frequency, object size, concurrency, and memory. Immutability is a design tool, not a command to copy enormous graphs on every event.

Immutable Collections and Null Semantics

Factory methods such as `List.of`, `Set.of`, `Map.of`, and `copyOf` produce unmodifiable collections and reject null elements or values.

```
List<String> names = List.of("Alice", "Bob");
names.add("Carol"); // UnsupportedOperationException
```

`Collections.unmodifiableList` creates a wrapper around an existing list. It prevents mutation through the wrapper but reflects changes through another reference to the original.

```
List<String> source = new ArrayList<>(List.of("Alice"));
List<String> view = Collections.unmodifiableList(source);

source.add("Bob");
view.size(); // 2
```

`List.copyOf(source)` represents an unmodifiable snapshot instead.

Document null policy and snapshot-versus-view semantics in public APIs. An unmodifiable collection is not necessarily deeply immutable, and an immutable collection implementation may not guarantee object identity across calls.

Caching Derived Values

An immutable object's derived value can be cached because its inputs do not change.

```
public final class Document {

    private final String content;
    private volatile String sha256;

    public String sha256() {
        String result = sha256;

        if (result == null) {
            result = calculateSha256(content);
            sha256 = result;
        }

        return result;
    }
}
```

The object is logically immutable even though it has an internal memoization field, because callers observe the same derived value. The volatile field provides publication for concurrent readers.

This is sometimes called observational immutability. It should be used carefully: computation must be deterministic, the cached object must not be mutable, and failure behavior must be defined. Often eager calculation is simpler unless profiling shows a meaningful cost.

Serialization, Reflection, and Framework Boundaries

Frameworks may construct objects through reflection, deserialize fields without ordinary constructors, or require no-argument constructors and mutable properties. These mechanisms can bypass the invariants an immutable type normally protects.

Prefer serialization libraries that support constructors or records and validate input during creation. Treat deserialized data as untrusted and re-establish invariants.

Do not expose mutable internal collections merely to satisfy a mapper. Use dedicated transport DTOs when framework requirements conflict with the domain model:

```
JSON payload
     |
     v
mutable or framework-friendly DTO
     |
validate and convert
     v
immutable domain object
```

Java serialization also deserves caution because it can restore object state without following ordinary construction semantics and creates security and compatibility risks. Prefer explicit formats and controlled reconstruction.

Reflection can technically mutate fields under some circumstances, but this does not make immutability meaningless. The contract assumes callers use supported APIs and trusted runtime configuration. Strong

module boundaries and minimal reflective access improve integrity.

Immutability and Domain Entities

Not every domain object should be immutable. An Order entity may have a lifecycle with meaningful transitions such as submit, approve, and cancel. Modeling it as mutable can be appropriate when one aggregate owns those transitions and protects its invariants.

Even then, immutable components are valuable:

- OrderId
- Money
- Address snapshot
- LineItem value
- Status-change event
- Audit timestamp

An entity can own controlled mutable state while composing immutable values. This reduces the surface area requiring synchronization, copying, and defensive reasoning.

The question is not "Can everything be immutable?" It is "Which state represents a value or snapshot, and who owns each mutation?"

Testing Immutability

Tests should try to mutate through every boundary.

```
@Test
void scheduleDefensivelyCopiesInput() {
    List<Instant> source = new ArrayList<>();
    source.add(first);

    Schedule schedule = new Schedule(source);
    source.add(second);

    assertEquals(List.of(first), schedule.times());
}
```

Verify output protection:

```
@Test
void scheduleDoesNotExposeMutableState() {
    Schedule schedule = new Schedule(List.of(first));

    assertThrows(
        UnsupportedOperationException.class,
        () -> schedule.times().add(second)
    );
}
```

For arrays, verify that changing the input and returned arrays does not change equality or content. For builders, verify that builder reuse does not alter already built objects. For nested structures, test element mutation or use immutable element types.

Concurrency tests may support evidence, but the primary proof should come from design: no mutation path, validated final state, no constructor escape, and safe publication.

Production Perspective

Immutability failures often appear as action at a distance. A controller adds an item to a list and unexpectedly changes a cached response. A builder is reused and mutates objects that were thought to be snapshots. A byte array used as a map key changes after insertion. A record contains a mutable list and its generated hash changes. A supposedly read-only view changes because another component retained the backing collection.

These failures are difficult because the mutation may occur in a different thread, layer, or request from the visible symptom.

When diagnosing them:

- Identify every reference to the mutable object, not only the containing field.
- Determine who owns mutation and whether the API promises a snapshot or live view.
- Inspect constructors and accessors for arrays, collections, and legacy mutable dates.
- Check nested element mutability.
- Look for builder reuse, deserialization bypass, reflection, and constructor escape.
- Verify that equality and hash fields cannot change.
- Use heap paths and thread traces when shared state changes unexpectedly.

Defensive copying consumes CPU and memory, but hidden shared mutation consumes engineering time and operational reliability. Copy at ownership boundaries where the safety benefit is real, then profile large-object paths.

Interview-Ready Summary

An immutable object does not change its observable state after construction. A robust immutable class validates all input, stores private final state, prevents constructor escape, defensively copies mutable inputs, and exposes no mutable internal representation.

Final fields prevent reference reassignment but do not freeze referenced objects. `List.copyOf` creates an unmodifiable snapshot, while `Collections.unmodifiableList` creates a read-only view whose backing collection can still change. Collection copies are shallow, so deep immutability also requires immutable or copied elements.

Arrays and legacy mutable dates normally require copies on both input and output. Records provide final components and generated value methods but are only shallowly immutable. Builders may be mutable, but the built object must copy builder-owned mutable state.

Correct construction and final fields support safe publication, but mutable objects reachable through final fields still require coordination. Immutable values simplify concurrency because readers do not synchronize changes.

Immutability is most valuable for value objects, identifiers, configuration, messages, events, and snapshots. Stateful entities can remain controlled aggregates while composing immutable values. The design decision follows ownership, lifecycle, update frequency, and the cost of copying.

Revision Notes

- Immutable objects do not change observable state after construction.
- Reassigning a variable does not mutate the old immutable object.
- Final fields prevent reassignment; they do not freeze referenced objects.
- Validate all invariants during construction.

- Do not allow `this` to escape during construction.
- Copy mutable inputs before storing them.
- Do not return mutable internal arrays or collections.
- `List.copyOf` creates an unmodifiable snapshot.
- `Collections.unmodifiableList` is a view and can reflect backing-list changes.
- Collection copying is shallow; mutable elements can still change observable state.
- Arrays normally require copies on input and output.
- Prefer immutable `java.time` values over mutable legacy Date types.
- Records are only shallowly immutable and may require compact-constructor copies.
- Builders must copy their mutable state into the constructed value.
- Final-field guarantees require correct construction and no premature publication.
- Functional updates return new values rather than modifying existing ones.
- Persistent collections can reduce the cost of large immutable updates.
- Memoization can preserve observational immutability when derived values are stable and safely published.
- Validate framework-deserialized data and separate DTOs from domain objects when necessary.
- Use immutable values inside controlled mutable entities to reduce the mutation surface.
- Test input copying, output protection, nested elements, arrays, and builder reuse.